

Clase 1

Criterios para evaluar lenguajes

Simplicidad y Legibilidad

- Facilidad de entender y utilizar el lenguaje. Un lenguaje simple tiende a tener menos reglas y conceptos complicados.
- Facilidad para leer y comprender el código escrito. Un lenguaje legible facilita la colaboración entre programadores y la mantención a lo largo del tiempo.
- Uno afecta al otro

Claridad en los bindings

- Facilidad con la que se lee la conexión entre objetos y sus atributos, propiedades o métodos. Lo mismo para bibliotecas y el programa.
- Debe ser claro en qué momento se hacen (compilación, ejecución)
- Ej: Cuando se asigna el tipo a una variable

Confiabilidad

- Capacidad del lenguaje para chequear errores lo antes posible, interceptar errores en tiempo de ejecución, tomar medidas correctivas y continuar.
- Chequeo de tipos, manejo de excepciones.

Soporte

- Disponibilidad de recursos y herramientas (documentación, comunidad activa, bibliotecas, cursos, tutoriales).
- Accesibilidad para cualquiera (uso o instalación).
- Ideal compilador o intérprete de dominio público.
- Debería poder ser implementado en distintas plataformas.

Abstracción

- Capacidad para ocultar detalles innecesarios que permita definir y utilizar estructuras u operaciones complicadas de forma sencilla.

Ortogonalidad

- Se refiere a la consistencia e independencia de los diferentes componentes del lenguaje. En un lenguaje ortogonal, las características se combinan de manera predecible y coherente, lo que facilita su comprensión y uso.
- Es por aspectos, C en cuanto a los returns no es ortogonal, porque, por ejemplo, se puede trabajar con arrays (definir, usar métodos) pero no se pueden retornar.

Eficiencia

- La eficiencia se refiere al rendimiento y al uso de recursos del lenguaje. Un lenguaje eficiente ejecutará programas rápidamente y utilizará la menor cantidad de memoria posible.

Lenguaje de programación

- Es una notación formal para describir algoritmos a ser ejecutados en una computadora. Como toda notación formal tiene una sintaxis y una semántica.

Sintaxis

- Conjunto de reglas que definen como componer letras, dígitos y otros caracteres para formar los programas
- Define cómo se deben combinar los elementos del lenguaje (palabras clave, operadores, identificadores) para formar expresiones y sentencias válidas.

Semántica

- Conjunto de reglas que nos sirven para darle significado a los programas sintácticamente válidos.

Características de la sintaxis

- Legibilidad
- Verificabilidad
- Traducción
- Falta de ambigüedad

La sintaxis establece reglas que definen cómo deben combinarse las “**words**”

- componente básico no elemental que se utiliza para formar sentencias y programas.

Elementos de la sintaxis

- Alfabeto o conjunto de caracteres
 - Hay que saber cuál se usa para no cometer errores sintácticos (existencia o no de la ñ).
- Identificadores
 - Nombres de las funciones y variables
- Operadores
- Palabras clave
 - Palabra que tiene un determinado sentido dentro de un contexto
- Palabras reservadas
 - Palabras clave que no pueden ser utilizadas como identificadores
- Comentarios y usos de blancos

Estructura sintáctica

- **Vocabulario o words**
 - Conjunto de words que construyen expresiones, sentencias y programas.
- **Expresiones**
 - Funciones que a partir de datos, devuelven un resultado.
 - Bloques sintácticos básicos que construyen sentencias y programas.
 - $5 + 3$ es una expresión que se evalúa como 8.
 - `len("Hola mundo")` es una expresión que devuelve la longitud de la cadena "Hola mundo".
 - `True and False` es una expresión que se evalúa como False.

- Sentencias

- Instrucción que realiza una acción.
- Componente sintáctico más importante.
- Alto impacto en la simplicidad y legibilidad.
- Pueden ser simples / estructuradas / anidadas.
- $x = 5$ es una sentencia que asigna el valor 5 a la variable x.
- `print("Hola mundo")` es una sentencia que imprime la cadena "Hola mundo".
- `if x > 0: ...` es una sentencia que inicia una estructura condicional que ejecuta cierto código si la condición $x > 0$ es verdadera.

Reglas léxicas

- Conjunto de reglas para formar las word. Nos determinan cómo se componen los caracteres para formar una word.

Reglas sintácticas

- Conjunto de reglas que definen cómo formar las expresiones y sentencias.

Tipos de sintaxis

Tipo	¿Qué analiza?	C	Pascal
		<code>while (x != x) { ... }</code>	<code>while x <> z do begin ... end;</code>
Abstracta	Estructura	Ambas comparten estructura <i>while condición bloque</i>	
Concreta	Parte léxica	Difieren en la sintaxis: - Uso de paréntesis - Forma de encerrar un bloque - Símbolo de distinto	
Pragmática	Legibilidad y practicidad.	Puede omitirse el “{}” o “begin end;” si el bloque está compuesto por una sola sentencia. Puede conducir a error si se agrega otra sentencia.	

Formas de definir la sintaxis:

- Lenguaje natural.
- Gramática libre de contexto: BNF, EBNF.
- Diagramas sintácticos

BNF (Backus Naun Form)

- Notación formal para describir la sintaxis.
- Metalenguaje → Porque se utiliza para definir la sintaxis de otros lenguajes.
- Utiliza metasímbolos
 - $< > ::= |$
- Define las reglas por medio de producciones.

Gramática

- Conjunto de reglas finito que define un conjunto infinito de posibles sentencias válidas en el lenguaje.
- Está formada por una 4-tupla
 - $G = (N, T, S, P)$
 - N = Conjunto de símbolos no terminales.
 - T = Conjunto de símbolos terminales.
 - S = Símbolo distinguido de la gramática. $S \in N$
 - P = Conjunto de producciones.

Árbol sintáctico o de derivación

- Es construido como objeto de un método de análisis que permite determinar si un string dado es válido o no en el lenguaje. Parsing
- Se puede construir de dos formas
 - Bottom-up
 - Top-down

Producciones recursivas

- Son las que hacen que el conjunto de sentencias descrito sea infinito.
- Cualquier gramática que tiene una producción recursiva describe un lenguaje infinito.
- $\langle \text{expresión} \rangle ::= \langle \text{id} \rangle \mid \langle \text{id} \rangle \langle \text{operador} \rangle \langle \text{expresión} \rangle$

Gramática ambigua

- Una gramática es ambigua si una sentencia puede derivarse de más de una forma.
- $\langle \text{expresión} \rangle ::= \langle \text{id} \rangle \mid \langle \text{id} \rangle \langle \text{operador} \rangle \langle \text{expresión} \rangle \mid \langle \text{expresión} \rangle \langle \text{operador} \rangle \langle \text{id} \rangle$
 - Tener ambos tipos de recursión (por izquierda y por derecha) hace ambigua a la gramática.

Subgramáticas

- Si se tiene una gramática para, por ejemplo, identificadores (GI) y otra para números (GN), estas podrían usarse para definir una gramática para expresiones (GE). A las primeras dos se las llamaría **subgramáticas** en este caso.

Gramáticas libre de contexto (BNF)

- Es aquella gramática que no realiza un análisis del contexto.
 - La expresión $a := b + c$; es sintácticamente válida, pero puede ser que no lo sea semánticamente (a, b o c pueden no ser del mismo tipo).

Gramáticas sensibles al contexto

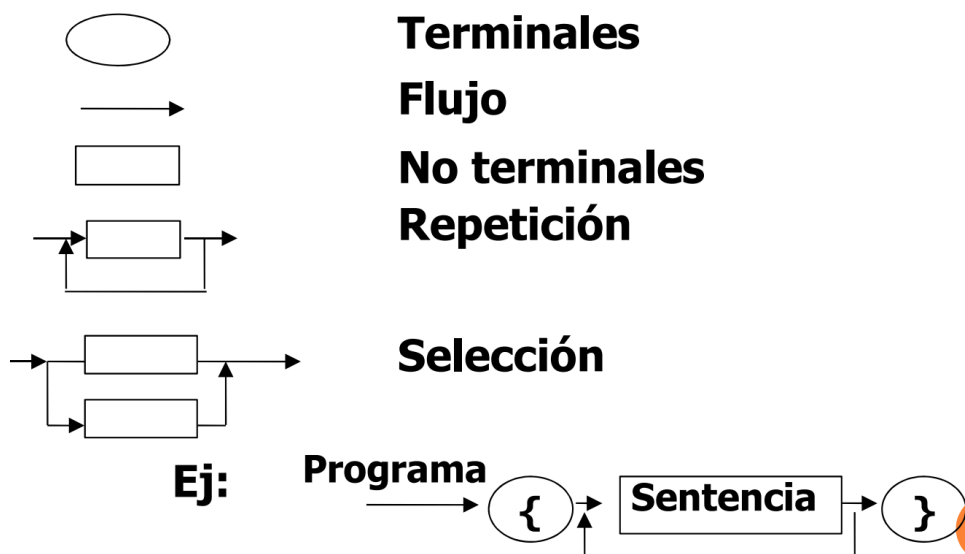
- Es aquella gramática que analiza el contexto, a través de la semántica estática (semántica que se analiza en tiempo de compilación).
- Analiza que en un mismo contexto/ámbito no ocurran problemas semánticos (que no haya dos o más variables con el mismo nombre, que no se opere con variables de distinto tipo, etc)

EBNF - Extended Backus Naum Form

- Se agregan los metasímbolos:
 - $[]$ = elemento opcional
 - $(|)$ = selección de una alternativa
 - $\{\}$ = repetición
 - $*$ = 0 o más veces
 - $+$ = 1 o más veces
- Elimina la recursión.
- Se necesitan menos producciones, y además, abarcan más casos.

Diagramas sintácticos o de CONWAY

- Forma gráfica de representar las producciones.
- Cada diagrama tiene una entrada y una salida.
- Cada diagrama representa una regla o producción.
- Para que una sentencia sea válida, debe haber un camino desde la entrada hasta la salida.
-



Clase 2

Semántica

- Describe el significado de los símbolos, palabras y frases de un lenguaje, ya sea lenguaje natural o lenguaje informático sintácticamente válido.

Estática

- Antes de la ejecución.
- No está relacionada con el significado de la ejecución del programa, está más relacionado con las formas válidas (con la sintaxis).
- El análisis está entre el análisis sintáctico y el de semántica dinámica.
- BNF / EBNF no sirve para esto.
- Se utiliza la gramática de atributos para describir la sintaxis.

Gramática de atributos

- Describe la sintaxis y la semántica estática formalmente.
- Es una gramática sensible al contexto.
- La usan los compiladores.

¿Cómo funciona?

- Asocia información a las construcciones del lenguaje a través de **atributos**, asociados a los símbolos de la gramática correspondiente.
- Los valores de los atributos se calculan mediante **ecuaciones** o **reglas semánticas** asociadas a las producciones gramaticales.
- Evaluar estas reglas puede:
 - Generar código.
 - Insertar información en la Tabla de Símbolos.
 - Realizar el Chequeo Semántico.
 - Dar mensajes de error.
- Las **reglas sintácticas** (producciones) son similares a BNF.
- Las **reglas semánticas** (ecuaciones) permiten detectar errores y obtener valores de atributos.
- Un atributo puede ser: valor de/tipo de/lugar que ocupa una variable, una expresión, dígitos significativos de un número, etc.
- Se desarrolla en forma de tabla
-

Regla gramatical	Reglas semánticas (obtener el valor)
Regla 1	Ecuaciones de atributo asociadas
.	.
.	.
Regla n	Ecuaciones de atributo asociadas

15

- **Mira las reglas** de BNF/EBNF y **busca atributos** para **terminales** y **no terminales**, si **encuentra el atributo** debe llegar a **obtener su valor**, esto lo logra **generando ecuaciones**, **usa la tabla** donde **machea** si **encuentra la producción/regla** y del otro lado están **las ecuaciones que me permiten llegar a los atributos**.
- De la ejecución de las ecuaciones:

- Detecta errores y genera mensajes de error.
- Se ingresan símbolos a la tabla de símbolos.
- Detecta dos variables iguales.
- Controla tipo y variables de igual tipo.
- Controla reglas específicas del lenguaje.
- Genera un código para el siguiente paso.

Dinámica

- Durante la ejecución.
- Describe el significado de ejecutar las diferentes construcciones del lenguaje de programación.
- Su efecto se ve durante la ejecución del programa.
- Es complejo describirla con notaciones formales, no existen herramientas estándares.

Soluciones

- Formales complejas:
 - Semántica axiomática
 - Semántica denotacional
- Informal
 - Semántica operacional

Semántica axiomática

- Considera al programa como una **máquina de estados** donde cada instrucción provoca un cambio de estado.
- Se parte de un **axioma** (verdad) que sirve para verificar **estados y condiciones** a probar.
- Se desarrolló para probar la corrección de los programas.
- Se utiliza la notación **cálculo de predicados**.
- Un **estado** se describe con un **predicado**.
- El **predicado** describe los valores de las variables en ese estado.
- Existe un **estado anterior** y un **estado posterior** a la ejecución del constructor.

Semántica denotacional

- Se basa en la teoría de **funciones recursivas** y **modelos matemáticos**, es más exacto para obtener y verificar resultados, pero es más difícil de leer.
- Describe los estados a través de funciones (recursivas).
- Define una correspondencia entre los constructores sintácticos y sus significados.
- Lo que hace es buscar funciones que se aproximen a las producciones sintácticas.

Semántica operacional

- El **significado de un programa** se describe **mediante otro lenguaje de bajo nivel** implementado sobre una máquina abstracta.
- Cuando se ejecuta una sentencia del lenguaje de programación los cambios de estado de la máquina abstracta definen su significado.
- Es un método informal porque se basa en otro lenguaje de bajo nivel y puede llevar a errores.
- Es el más utilizado en los libros.

Procesamiento de un programa

- En los inicios se programaba en código máquina (0s y 1s), esto era muy complejo y traía muchos errores. Debido a esto surge el **lenguaje ensamblador**, que utilizaba **códigos mnemotécnicos**. El problema era que cada máquina tenía su propio set de instrucciones, esto hacía imposible el intercambio de programas entre distintas máquinas o familias de procesadores (diferentes versiones de un cpu podría tener distinto set de instrucciones). Para solucionar esto nacen los lenguajes de alto nivel, pero aún se necesitaba algún tipo de programa traductor que genere el código de bajo nivel.
- Soluciones (el desarrollador del lenguaje elige):
 - Interpretación
 - Compilación
 - Interpretación y compilación

Interpretación

- Existe un programa escrito en lenguaje de programación interpretado.
 - Lisp, Smalltalk, Basic, Python, Ruby, PHP, Perl.
- Existe un programa llamado **intérprete**, que realiza la traducción de ese lenguaje de la siguiente forma:
 1. Lee
 2. Analiza
 3. Decodifica
 4. Ejecuta una a una las sentencias del programa.
- Solo pasa por ciertas líneas, determinado por el flujo del programa.
- Cada vez que se ejecuta el programa se realiza el proceso.
- El intérprete cuenta con subprogramas que ayudan a la traducción.
 - Por cada posible acción, hay un subprograma en lenguaje máquina que ejecuta esa acción.
 - La interpretación se realiza llamando a estos subprogramas en la secuencia adecuada hasta generar el resultado de la ejecución.
- Un intérprete ejecuta repetidamente:
 1. **Obtiene** la próxima sentencia.
 2. **Determina** la acción a ejecutar.
 3. **Ejecuta** la acción.

Compilación

- Existe un programa escrito en lenguaje de alto nivel no interpretado.
 - Ada, C++, Fortran, Pascal
- Existe un programa llamado **compilador**, que realiza la traducción de ese lenguaje a lenguaje de máquina.
- Se traduce/compila antes de la ejecución.
- Pasa por todas las instrucciones antes de la ejecución.
- El código que se genera se guarda y se puede reusar ya compilado.
- La compilación implica las **etapas de análisis y de síntesis**.
- El compilador toma todo el programa escrito en **lenguaje de alto nivel**, también llamado **lenguaje fuente**.
- Luego de la compilación, genera un **lenguaje objeto**, que es generalmente el ejecutable (en código máquina), o un **lenguaje de nivel intermedio**, el **lenguaje ensamblador**.

Etapas de Análisis

- Análisis léxico
- Análisis sintáctico
- Análisis semántico

Etapas de Síntesis

- Optimización del código
- Generación del código

Análisis léxico (Scanner)

- Es un proceso que lleva tiempo.
- Hace el análisis a nivel de palabra (LEXEMA) .
- Divide el programa en sus elementos/categorías: identificadores, delimitadores, símbolos especiales, operadores, números, palabras clave, palabras reservadas, comentarios, etc.
- Analiza el tipo de cada uno para ver si son TOKENS válidos.
- Filtra comentarios y separadores.
- Lleva una tabla para la especificación del analizador léxico. Incluye cada categoría, el conjunto de atributos y acciones asociadas.
- Pone los identificadores en la tabla de símbolos. Reemplaza cada símbolo por su entrada en la tabla.
- Genera errores si la entrada no coincide con ninguna categoría léxica.
- El resultado de este paso será el descubrimiento de los ítems léxicos o tokens y detección de errores.

Análisis sintáctico (Parser)

- El análisis se realiza a nivel de sentencia/estructuras.
- Usa los tokens del analizador léxico.
- Tiene como objetivo encontrar las estructuras presentes en su entrada.

- Estas estructuras se pueden representar mediante el árbol de análisis sintáctico.
- Se identifican las estructuras de las sentencias, declaraciones, expresiones, etc. ayudándose con los tokens.
- Se alterna/interactúa con el análisis léxico y análisis semántico.
- Usan técnicas basadas en gramáticas formales.

Análisis Semántico (Semántica estática)

- Debe pasar antes bien por Scanner y Parser.
- Es una de las fases más importantes.
- Procesa las estructuras sintácticas (reconocidas por el analizador sintáctico).
- Agrega información implícita.
- La estructura del código ejecutable continúa tomando forma.
- Realiza la comprobación de tipos (aplica gramática de atributos) .
- Agrega a la tabla de símbolos los descriptores de tipos.
- Realiza comprobaciones de duplicados, problema de tipos, etc.
- Realiza comprobaciones de nombres (todas las variables deben estar declaradas).

Etapas de Síntesis

- Vinculada a características del código objeto, del hardware y la arquitectura.
- Construye el programa ejecutable y genera el código necesario.
- Se genera el módulo de carga. Programa objeto completo.
- Se realiza el proceso de optimización:
 - Es Optativo.
 - Los optimizadores de código (programas) pueden ser herramientas independientes, o estar incluidas en los compiladores e invocarse por medio de opciones de compilación.